



MALAWI UNIVERSITY OF BUSINESS AND APPLIED SCIENCES

SCHOOL OF SCIENCE AND TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION SYSTEMS(COSISS)

BACHELOR OF SCIENCE IN MANAGEMENT INFORMATION SYSTEMS (BIS)

YEAR 4

PROJECT PROPOSAL

CIS-PRJ-411 RESEARCH PROJECT

**DEVELOPMENT OF A PROTOTYPE NETWORK MONITORING APPLICATION FOR
FAULT DETECTION AND MANAGEMENT AT MUBAS**

FROM: TUMWITIKE MAPOMA (BIS/22/SS/018)

SUBMITTED TO: Dr MARTIN MSENDEMA

DATE

17th APRIL 2026

1. INTRODUCTION	3
1.1. Background of the Study	3
1.2. Problem Statement	4
1.3. Aims and Objectives	4
1.3.1. Aim of the Study	5
1.3.2. Objectives of the Study	5
1.4. Research Question	5
2. LITERATURE REVIEW	5
2.1. Discussion on Key Concepts	5
2.1.1. Network Monitoring Systems.....	6
2.1.2. Network Congestion and Failures	6
2.1.3. Data Analytics in Network Management	7
2.1.4. Application Development for Network Monitoring.....	7
2.1.5. Network Simulation Tools.....	7
2.2. Gap in Existing Studies.....	8
3. METHODOLOGY	8
3.1. Research Approach	8
3.2. System Design.....	8
3.2.1. Simulated Network Environment (Cisco Packet Tracer)	9
3.2.2. Prototype Monitoring Application.....	9
3.3. Tools and Technologies.....	10
3.4. Data Collection Methods.....	10
3.4.1. Device Failure	10
3.4.2. Link Failure.....	10
3.4.3. Network Congestion	11
3.5. Data Collection Variables	11
3.6. Data Analysis Techniques	11
3.7. Experimental Procedure	12
4. Ethical Considerations.....	12
5. Justification of the Study	12
References	14
6. APPENDICES: RESEARCH PLAN	15
6.1. APPENDIX A: BUDGET	15
6.2. APPENDIX B: WORK PLAN.....	15

Development of a Prototype Network Monitoring Application for Fault Detection and Management at MUBAS

1. INTRODUCTION

Reliable internet connectivity is a cornerstone for higher learning institutions, supporting essential functions such as teaching, learning, research, and administrative activities. As more tasks and resources move to digital platforms, the demand for stable and efficient campus network services continues to rise (Cisco, n.d.). Network monitoring has typically involved manual system checks with tools like Cacti, which offers visualisations of network usage and device status but does not automatically diagnose issues or suggest solutions (Lee et al., 2014). This reactive approach often delays the identification and resolution of network issues, allowing failures and congestion to persist for extended periods.

This study proposes the design and development of a prototype network monitoring application that goes beyond what Cacti currently offers. The prototype will provide real-time network status reporting, automated fault detection, fault diagnosis, and recommended corrective actions all tested against a simulated network environment built in Cisco Packet Tracer.

1.1. Background of the Study

Network monitoring is vital for maintaining the performance and reliability of communication systems. According to Cisco (n.d.), network monitoring involves the continuous observation of a network to detect performance degradation, failures, and anomalies in real time. Many organisations and educational institutions utilise network management tools to observe network status and performance.

At the Malawi University of Business and Applied Sciences (MUBAS), network monitoring is currently carried out using Cacti an open-source tool that collects and graphs network performance data such as bandwidth usage and device availability. Cacti shows historical and live usage data, but administrators must log in and analyze the information themselves to detect faults. It lacks automatic alerts, problem identification, diagnosis,

and action recommendations(*Cacti® - The Complete RRDTool-Based Graphing Solution*, n.d.).

High-traffic areas on campus, such as hostels, laboratories, and libraries, are particularly susceptible to network congestion and service interruptions. SolarWinds (n.d.) defines network congestion as a condition in which demand for network resources exceeds available capacity, leading to performance degradation. When such conditions arise, the manual nature of Cacti monitoring means faults may go undetected for extended periods, reducing academic productivity and user experience.

Data analysis techniques provide the capability to examine network usage patterns and performance trends, helping to identify potential issues before they manifest as failures (Lakshminarayana & Yau, 2018). Furthermore, network simulation tools like Cisco Packet Tracer offer a controlled environment for testing and evaluating network behaviour under various conditions (Cisco Packet Tracer, n.d.). Combining application development, data analysis, and simulation-based testing presents an opportunity to significantly improve how network faults are detected and managed at MUBAS.

1.2. Problem Statement

MUBAS uses Cacti for network monitoring, which requires manual checks. Administrators must log in, review graphs, and identify faults themselves. The tool does not provide automatic alerts, fault location, cause analysis, or resolution guidance (Lee et al., 2014).

The absence of automated detection, diagnosis, and advisory capabilities means that network faults and congestion may persist for extended periods before corrective action is taken. This results in increased network downtime, reduced service reliability, and a negative impact on academic and administrative activities (Tanenbaum & Wetherall, 2011).

A prototype monitoring app is needed to address Cacti's limitations by offering automated real-time fault detection, location identification, problem diagnosis, and recommended actions to solve the fault in a user-friendly interface.

1.3. Aims and Objectives

1.3.1. Aim of the Study

This study aims to develop a prototype network monitoring app offering real-time status, automated fault detection, diagnosis, and suggested actions, improving the features of the current Cacti tool at MUBAS.

1.3.2. Objectives of the Study

- a) To design a simulated network environment using Cisco Packet Tracer to serve as the testing infrastructure for the prototype application.
- b) To develop a prototype network monitoring application capable of reading real-time network status from the simulated network environment.
- c) To implement automated fault detection and reporting module within the prototype that identifies the type of network fault and solution.
- d) To evaluate the prototype by comparing its fault detection and response performance against the manual monitoring currently used at MUBAS.

1.4. Research Question

Based on the aim of the study, the following main research question guides this investigation:

How can a network monitoring prototype with real-time reporting, automated fault detection, diagnosis, and recommendations improve network management versus manual monitoring at MUBAS?

2. LITERATURE REVIEW

This chapter reviews existing literature related to network monitoring tools, network congestion, data analytics in network management, application development for network monitoring, and the use of simulation tools. The purpose of this review is to provide a theoretical foundation for the study, identify gaps in existing tools such as Cacti, and justify the need for a prototype application that integrates automated fault detection and advisory capabilities.

2.1. Discussion on Key Concepts

2.1.1. Network Monitoring Systems

Network monitoring systems provide network administrators with real-time data necessary to assess whether a network is performing at an optimal level (Cisco, n.d.). Traditional monitoring tools such as Cacti and Nagios allow administrators to view network statistics, including bandwidth usage, device status, and uptime through graphical dashboards.

Cacti, specifically, is an open-source network monitoring tool that uses the Round Robin Database Tool (RRDTool) to collect and graph time-series performance data from network devices via SNMP (Cacti Group, n.d.). While it is widely used and effective for visualising network trends, it does not provide automated alerting, fault localisation, or diagnostic guidance. Administrators must interpret the data manually, which introduces delays in fault response (Lee et al., 2014).

More advanced systems, such as Zabbix and PRTG Network Monitor, incorporate automated alerting and some level of event correlation. However, these tools are complex, resource-intensive, and may not be practically accessible for all institutions (Vyncke & Paggen, 2007). This study aims to develop a lightweight prototype that captures the most critical enhancements the real-time status, automated fault detection, diagnosis, and recommendations in a simple, accessible application.

2.1.2. Network Congestion and Failures

Network congestion occurs when the demand for network resources exceeds available capacity, resulting in reduced performance, increased latency, and packet loss (SolarWinds, n.d.). Congestion frequently occurs in environments with high demand, such as universities, where numerous users simultaneously access network resources.

Network failures may also arise due to device malfunction, link disruption, or configuration errors. According to Tanenbaum and Wetherall (2011), both congestion and failures significantly impact network performance and user experience. In campus environments, such issues can disrupt online learning, research, and communication. Forouzan (2012) further notes that without automated monitoring mechanisms, fault persistence is prolonged, reducing the overall quality of service experienced by end users.

2.1.3. Data Analytics in Network Management

Data analytics helps improve network management by allowing administrators to spot patterns, trends, and issues in performance data (Lakshminarayana & Yau, 2018). Techniques such as threshold-based alerting, trend analysis, and statistical modelling enable proactive fault management.

The integration of data analytics into network monitoring applications enhances their ability to detect unusual behaviour and support faster response. However, tools like Cacti do not fully utilise these capabilities, as they display raw data without interpretation or actionable recommendations. This gap represents a key motivation for the prototype developed in this study.

2.1.4. Application Development for Network Monitoring

Custom network monitoring applications can be developed to address the specific limitations of general-purpose tools. Python, for example, provides libraries such as pysnmp for SNMP-based device polling, and frameworks such as Flask or Django for building web-based dashboards (Lutz, 2013). Such applications can be tailored to specific institutional needs and integrated with simulation environments for testing purposes.

Sommerville (2016) highlights that iterative prototyping is an effective development approach for systems where requirements are still being refined. In this study, an iterative approach will be used to develop and test the prototype against a Cisco Packet Tracer simulated network, allowing continuous refinement based on observed performance.

2.1.5. Network Simulation Tools

Network simulation tools provide a controlled environment for designing, testing, and evaluating network performance. Cisco Packet Tracer is one of the most widely used simulation tools for educational and research purposes (Cisco Packet Tracer, n.d.). It allows users to create virtual network topologies, simulate network traffic, and observe how networks behave under different conditions.

Cisco Packet Tracer will be used as the simulated environment to test the prototype monitoring application. Simulated faults including device failure, link failure, and network

congestion will be introduced, and the application's ability to detect, diagnose, and suggest responses will be assessed.

2.2. Gap in Existing Studies

While previous studies have explored network monitoring, congestion management, and data analytics, there is limited work on lightweight prototype applications specifically designed to add automated fault detection and advisory capabilities on top of passive monitoring tools like Cacti. Most existing research either evaluates commercial enterprise tools or proposes theoretical frameworks without practical implementation (Lee et al., 2014).

3. METHODOLOGY

3.1. Research Approach

The study adopts a quantitative research approach combined with a prototyping development methodology (Creswell, 2014; Sommerville, 2016). The quantitative component focuses on measuring network performance indicators such as fault detection time and recovery time. The prototyping methodology allows the application to be built, tested, and refined iteratively against the simulated network environment.

3.2. System Design

The study involves two components: a simulated network environment built in Cisco Packet Tracer, and a prototype monitoring application developed in Python. Together, these form an integrated system where the application reads network status from the simulation and responds to faults automatically.

3.2.1. Simulated Network Environment (Cisco Packet Tracer)

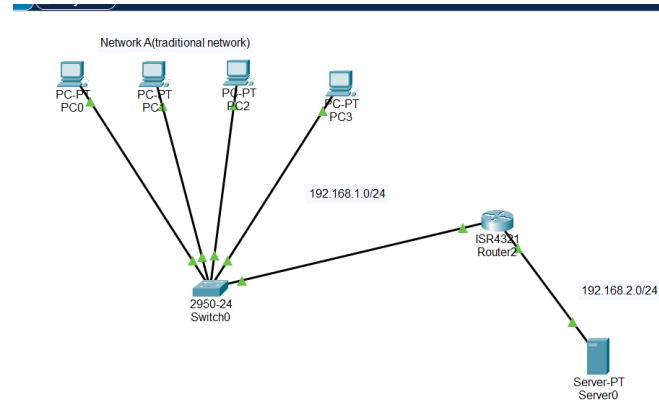


Figure 1: network design

All devices will be assigned static IP addresses in the 192.168.1.0/24 and the server will use 192.168.2.0/24 subnet. SNMP will be enabled on the router and switch to allow the prototype application to poll device status. The simulation will be used to introduce controlled fault scenarios including device failure, link failure, and network congestion.

3.2.2. Prototype Monitoring Application

The prototype application will be developed in Python and will consist of the following modules:

- Network Status Module:** Polls all network devices every 30 seconds using SNMP GET requests via the pysnmp library. Displays real-time device availability and response times on a dashboard.
- Fault Detection Module:** Monitors polling responses and triggers a fault alert if a device fails to respond within 5 seconds. Records the timestamp, affected device, and fault type automatically.
- Fault Diagnosis Module:** Analyses the pattern of unresponsive devices to determine the likely cause, for example, if only one PC is unreachable, a link failure is diagnosed; if the router is unresponsive, a device failure is flagged; if all devices show high response times, congestion is identified.
- Recommendation Module:** Based on the diagnosed fault type, the application presents a specific corrective recommendation to the administrator.

- e) Notification Module: Sends an in-application alert notification to the administrator immediately when a fault is detected, without requiring them to be actively monitoring the dashboard.
- f) Reporting Module: Logs all network events, fault detections, and responses to a CSV file for historical analysis and generates summary reports on request.

3.3. Tools and Technologies

- a) Cisco Packet Tracer
- b) Python 3.x
- c) Flask (Python web framework)
- d) HTML/CSS/JavaScript
- e) CSV flat-file storage
- f) Microsoft
- g) Personal Computer

3.4. Data Collection Methods

Data will be collected through simulation-based experimentation. The prototype application will be run against the Cisco Packet Tracer network while the following fault scenarios are introduced:

3.4.1. Device Failure

- A router or switch is powered off in Cisco Packet Tracer
- The application records the time from fault introduction to automated detection and alert generation
- For comparison, an administrator manually monitoring Cacti-style graphs records how long it takes to notice the same fault

3.4.2. Link Failure

- A network cable is disconnected between devices in the simulation
- Detection and diagnosis times are recorded for both the prototype application and the manual approach

3.4.3. Network Congestion

- Multiple PCs simultaneously send continuous traffic to the server
- Packet delay and packet loss rates are measured, the application's congestion diagnosis and recommendation are evaluated for accuracy

Trials will be conducted per scenario to ensure reliable and consistent measurements.

3.5. Data Collection Variables

- a) Fault Detection Time (seconds): Time from fault introduction to alert generation by the prototype, versus time for manual detection
- b) Fault Diagnosis Accuracy (%): Whether the application correctly identified the fault type across all trials
- c) Recovery Time (seconds): Time taken to restore normal network operation after a fault
- d) Packet Loss Rate (%): Percentage of packets lost during congestion scenarios
- e) Average Response Time (ms): Mean ping response time during normal and congested conditions

3.6. Data Analysis Techniques

The collected data will be analysed using the following statistical and analytical methods:

- a) Mean and Median: To summarise central tendency of detection and recovery times across trials
- b) Standard Deviation: To measure the consistency and variability of performance between the prototype and the manual approach
- c) Minimum and Maximum Values: To identify the range of performance outcomes
- d) Independent Samples T-Test: To determine whether the difference in mean fault detection times between the prototype application and manual monitoring is statistically significant ($\alpha = 0.05$)
- e) Frequency Analysis: To evaluate how consistently the fault diagnosis module correctly identified fault types

- f) Bar and Line Graphs: To visually compare performance across all scenarios

3.7. Experimental Procedure

- a) Design and implement the simulated network in Cisco Packet Tracer
- b) Develop and configure the prototype monitoring application
- c) Test the network and application under normal conditions and record baseline data
- d) Introduce each fault scenario (device failure, link failure, congestion) — 10 trials each
- e) Record fault detection time, diagnosis result, and recommendation output for each trial
- f) Repeat the same fault scenarios with a manual monitoring approach (simulating Cacti usage) and record detection times
- g) Export all data to Excel for statistical analysis
- h) Compare results using descriptive statistics and t-tests and document findings

4. Ethical Considerations

The study will use only simulated data from Cisco Packet Tracer, with no real user or institutional network information involved, ensuring privacy and ethical compliance. Interviews with MUBAS IT administrators will be voluntary, with informed consent, and all responses will be used anonymously for academic purposes.

5. Justification of the Study

The justification for this study stems from the clear limitations of the network monitoring tool currently in use at MUBAS. Cacti, while functional for displaying network data, does not automatically detect faults, identify their location, explain their cause, or advise on corrective actions. This places a heavy burden on administrators and increases the time taken to resolve network issues (Lee et al., 2014).

By developing a prototype application that addresses these gaps, this study contributes a practical and locally relevant solution to network management at MUBAS. The prototype

can serve as a foundation for future development into a fully deployable monitoring system tailored to the institution's infrastructure (Sommerville, 2016).

The study is also academically significant as it demonstrates how software development, data analytics, and network simulation can be combined to produce a functional system that solves a real institutional problem. It contributes to the growing body of literature on network management solutions for resource-constrained environments in sub-Saharan Africa (Tanenbaum & Wetherall, 2011).

References

- Cacti Group. (n.d.). About Cacti. Cacti. Retrieved April 2026, from <https://www.cacti.net/>
- Cisco. (n.d.). What is network monitoring? Cisco. Retrieved March 27, 2026, from https://www.cisco.com/c/en_uk/solutions/automation/what-is-network-monitoring.html
- Cisco Packet Tracer. (n.d.). Cisco Packet Tracer. Cisco Networking Academy. Retrieved March 27, 2026, from <https://www.netacad.com/cisco-packet-tracer>
- Creswell, J. W. (2014). Research design: Qualitative, quantitative, and mixed methods approaches (4th ed.). SAGE Publications.
- Forouzan, B. A. (2012). Data communications and networking (5th ed.). McGraw-Hill.
- Lakshminarayana, S., & Yau, D. K. Y. (2018). Cost-benefit analysis of Moving-Target defense in power grids. Proceedings — 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks, DSN 2018, 139–150. <https://doi.org/10.1109/DSN.2018.00026>
- Lee, S., Levanti, K., & Kim, H. S. (2014). Network monitoring: Present and future. Computer Networks, 65, 84–98. <https://doi.org/10.1016/j.comnet.2014.03.007>
- Cacti® - The Complete RRDTool-based Graphing Solution. (n.d.). Retrieved April 16, 2026, from <https://www.cacti.net/>
- Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media.
- SolarWinds. (n.d.). What is network congestion? SolarWinds IT Glossary. Retrieved March 27, 2026, from <https://www.solarwinds.com/resources/it-glossary/network-congestion>
- Sommerville, I. (2016). Software engineering (10th ed.). Pearson Education.
- Tanenbaum, A. S., & Wetherall, D. J. (2011). Computer networks (5th ed.). Pearson Prentice Hall.
- Vyncke, E., & Paggen, C. (2007). LAN switch security: What hackers know about your switches. Cisco Press.

6. APPENDICES: RESEARCH PLAN

6.1. APPENDIX A: BUDGET

Appendix A presents the project budget.

Item	Description	Cost (MWK)
Laptop/Computer	For development, simulation and analysis	Available
Internet Access	Research and downloading tools	20,000
Software	Cisco Packet Tracer, Python, Flask (all free)	0
Data Analysis Tools	Python / Microsoft Excel (Available)	0
Printing & Binding	Proposal and final report	20,000
Unforeseen Events	Contingencies	30,000
Total Estimated Cost		60,000 MWK

Table 1: Project Budget

6.2. APPENDIX B: WORK PLAN

The Gantt chart below outlines the timeline for the completion of all project activities. It shows the sequence and duration of each phase, including proposal development, literature review, system design, application development, simulation testing, data analysis, and report writing. This structured plan ensures that all activities are completed within the allocated time frame.

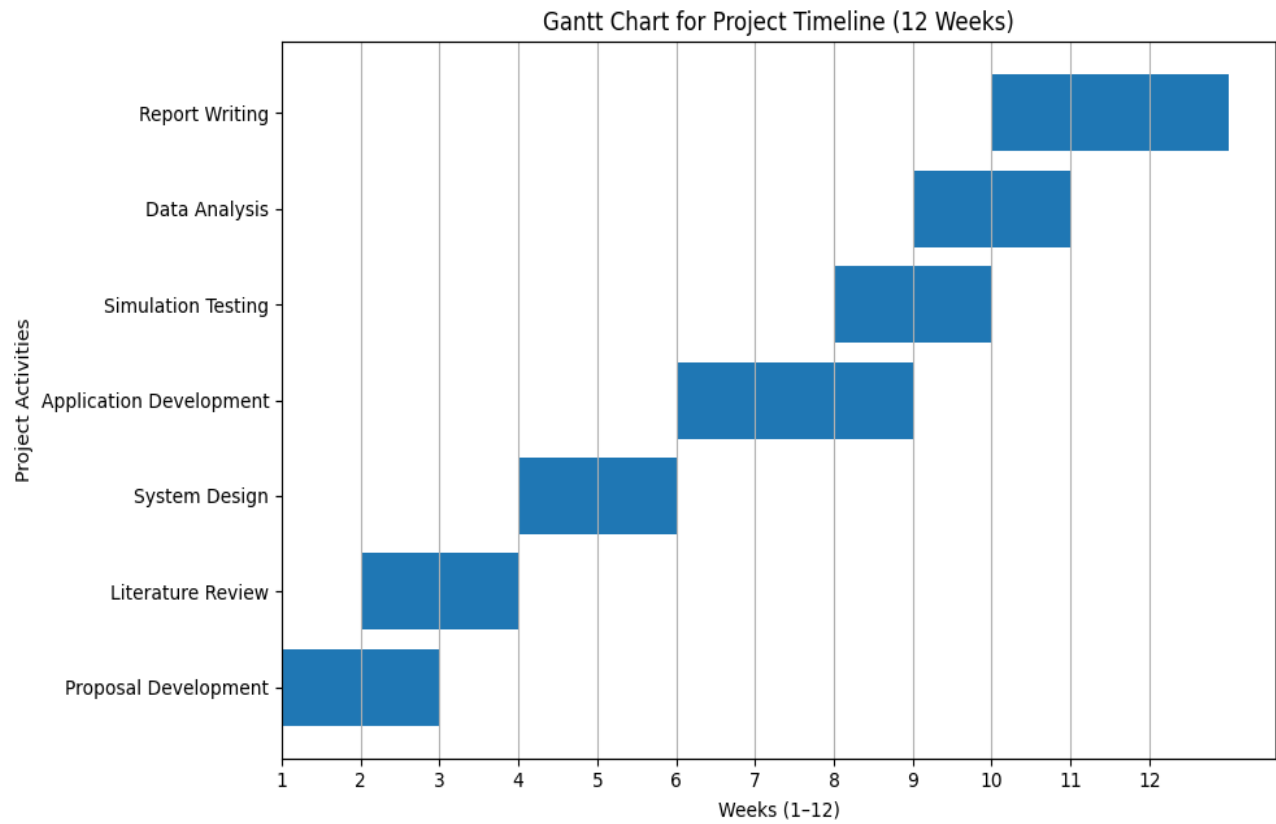


Figure 2: Project Work Plan (Gantt Chart)